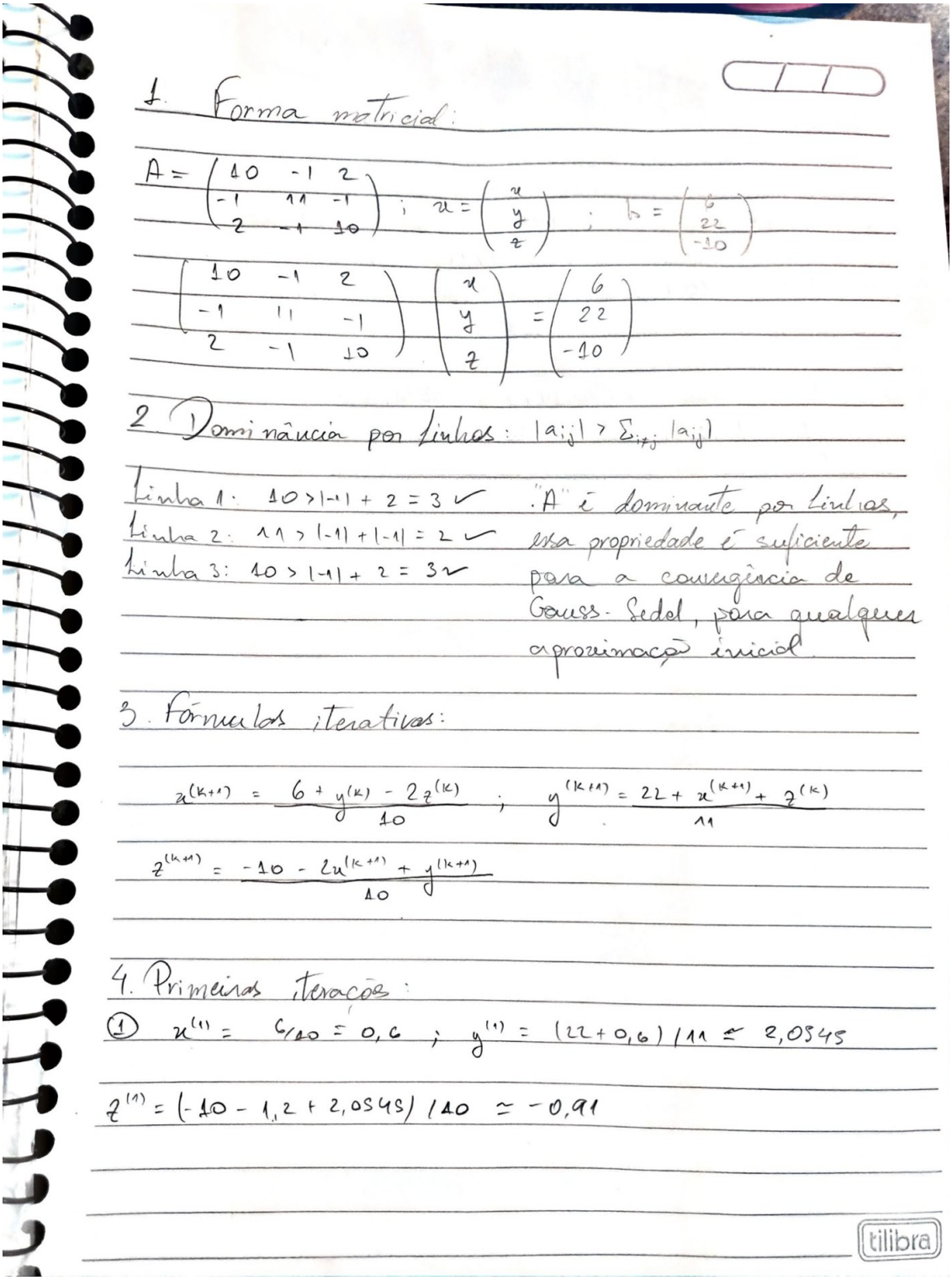
Cálculo Numérico – Lista de Exercícios

Sistemas lineares: método de Gauss–Seidel

Código no GitHub: https://github.com/Bensadon/Calculo-Numerico/blob/main/sistemas_lineares.ipynb

Parte manuscrita (questões 1 a 4)



1. Forma matricial:

$$A = \begin{pmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{pmatrix} ; \quad x = \begin{pmatrix} x \\ y \\ z \end{pmatrix} ; \quad b = \begin{pmatrix} 6 \\ 22 \\ -10 \end{pmatrix}$$

$$\begin{pmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 6 \\ 22 \\ -10 \end{pmatrix}$$

2. Dominância por linhas: $|a_{ij}| > \sum_{i \neq j} |a_{ij}|$

Linha 1: $10 > |-1| + 2 = 3$ ✓
Linha 2: $11 > |-1| + |-1| = 2$ ✓
Linha 3: $10 > |-1| + 2 = 3$ ✓

"A" é dominante por linhas, essa propriedade é suficiente para a convergência de Gauss-Seidel, para qualquer aproximação inicial

3. Fórmulas iterativas:

$$x^{(k+1)} = \frac{6 + y^{(k)} - 2z^{(k)}}{10} ; \quad y^{(k+1)} = \frac{22 + x^{(k+1)} + z^{(k)}}{11}$$

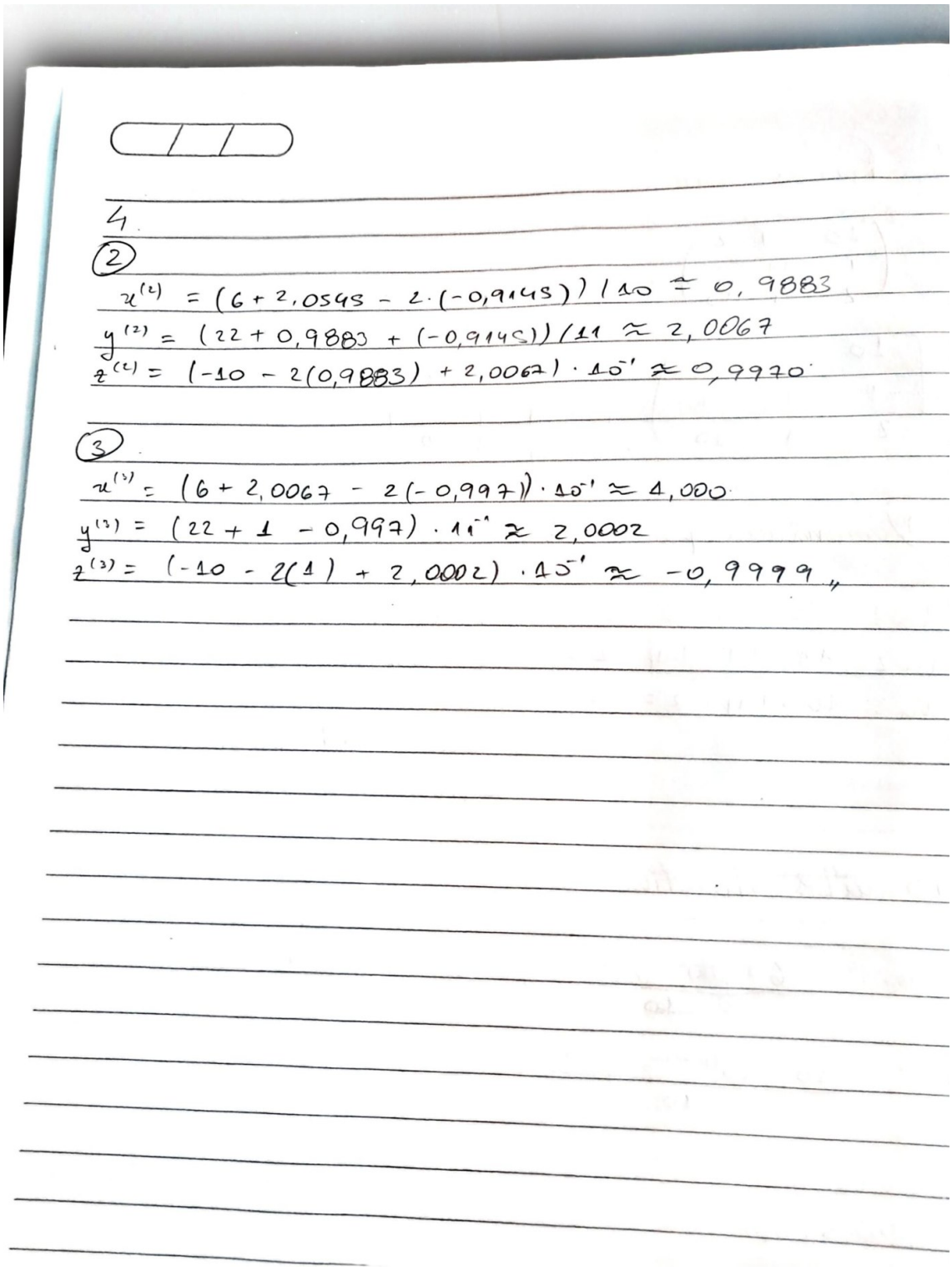
$$z^{(k+1)} = \frac{-10 - 2x^{(k+1)} + y^{(k+1)}}{10}$$

4. Primeiras iterações:

① $x^{(1)} = 6/10 = 0,6$; $y^{(1)} = (22 + 0,6)/11 \simeq 2,0545$

$z^{(1)} = (-10 - 1,2 + 2,0545)/10 \simeq -0,91$

Parte manuscrita (questão 4, continuação)



4.

(2)

$x^{(2)} = (6 + 2,0545 - 2\cdot(-0,9145))/10 \approx 0,9883$

$y^{(2)} = (22 + 0,9883 + (-0,9145))/11 \approx 2,0067$

$z^{(2)} = (-10 - 2(0,9883) + 2,0067)\cdot 10^{-1} \approx 0,9970$

(3)

$x^{(3)} = (6 + 2,0067 - 2(-0,997))\cdot 10^{-1} \approx 1,000$

$y^{(3)} = (22 + 1 - 0,997)\cdot 11^{-1} \approx 2,0002$

$z^{(3)} = (-10 - 2(1) + 2,0002)\cdot 10^{-1} \approx -0,9999$

Questão 5 – Implementação em Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

A = np.array([[10, -1,  2],
              [-1, 11, -1],
              [ 2, -1, 10]], dtype=float)
b = np.array([6, 22, -10], dtype=float)

def gauss_seidel(A, b, x0, tol=1e-5, max_iter=1000):
    n = len(b)
    x = x0.astype(float).copy()
    historico = [x.copy()]
    erros = [np.nan]                    # sem erro para k = 0
    for k in range(1, max_iter + 1):
        x_old = x.copy()
        for i in range(n):
            # x[:i] ja tem valores da iteracao k; x_old[i+1:] sao da k-1
            soma = A[i, :i] @ x[:i] + A[i, i+1:] @ x_old[i+1:]
            x[i] = (b[i] - soma) / A[i, i]
        erro = np.max(np.abs(x - x_old))  # norma infinito
        historico.append(x.copy())
        erros.append(erro)
        if erro < tol:
            return x, k, np.array(historico), np.array(erros)
    return x, max_iter, np.array(historico), np.array(erros)

x0 = np.zeros(3)
sol, iters, hist, erros = gauss_seidel(A, b, x0)

print("Iteracoes:", iters)
print("Solucao aproximada:", sol)
print("Solucao exata (np.linalg.solve):", np.linalg.solve(A, b))
```

Saída:

```
Iteracoes: 6
Solucao aproximada: [ 1.00000003  1.99999998 -1.00000001]
Solucao exata (np.linalg.solve): [ 1.  2. -1.]
```

O método convergiu em **6 iterações**, com solução aproximada (1,00000003; 1,99999998; –1,00000001), próxima da solução exata (1, 2, –1).

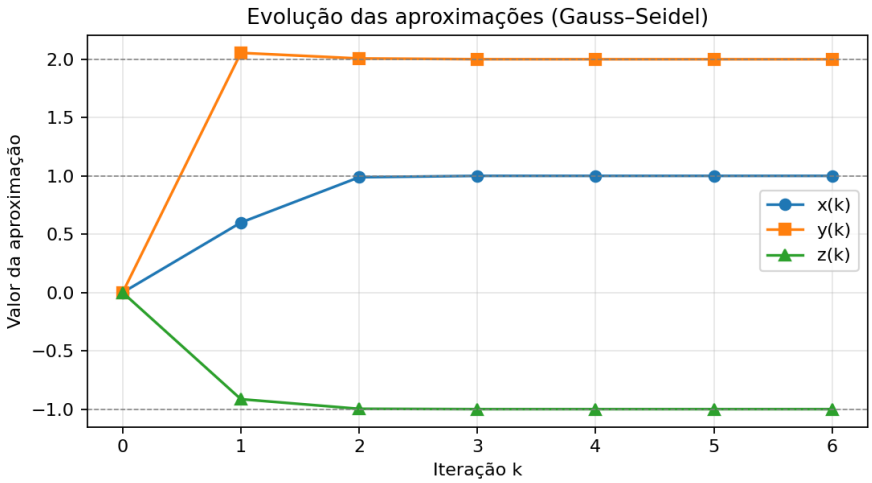
Questão 6 – Tabela das iterações

k	x(k)	y(k)	z(k)	||x(k) - x(k-1)||∞
0	0.00000000	0.00000000	0.00000000	—
1	0.60000000	2.05454545	-0.91454545	2.055e+00
2	0.98836364	2.00671074	-0.99700165	3.884e-01
3	1.00007140	2.00027907	-0.99998637	1.171e-02
4	1.00002518	2.00000353	-1.00000468	2.755e-04
5	1.00000129	1.99999969	-1.00000029	2.389e-05
6	1.00000003	1.99999998	-1.00000001	1.263e-06

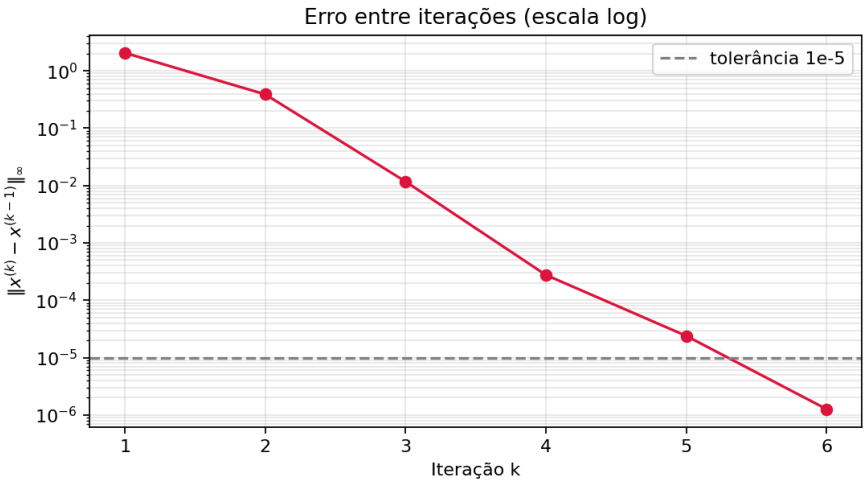
Tolerância 10^{-5}: o critério só é satisfeito em k = 6 (erro $1{,}263 \cdot 10^{-6}$); em k = 5 o erro ainda era $2{,}389 \cdot 10^{-5}$.

Questão 7 – Gráficos

(a) x(k), y(k) e z(k) em função de k



(b) Erro entre iterações em escala logarítmica



Código dos gráficos e da tabela: ver link do GitHub acima.